JNAN VIKAS MANDAL'S

Mohanlal Raichand Mehta College of Commerce
Diwali Maa College of Science
Amritlal Raichand Mehta College of Arts
Dr. R.T. Doshi College of Computer Science
NAAC Re-Accredited Grade 'A+' (CGPA : 3.31) (3rd Cycle)

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that Miss Sonali Thakur bearing seat no. 1314610 has done the project work/journal work in the subject of Blockchain of semester 4 Practical Examination during the academic year 2025-26 under the guidance of **Mrs. Sarita Sarang** being the partial requirement for the fulfillment of the curriculum of Degree in Master of Science in Information Technology under University of Mumbai.

Place: Airoli Date:

Sign of Subject in- Charge Sign of External Examiner

Sign of Coordinator

INDEX

Sr. No	Name	Date	Sign
1A	Develop a secure messaging system where users can exchange encrypted messages using RSA public-key cryptography.		
1B	Allow users to create multiple transactions and display them in an organised format.		
1C	Create a Python class named Transaction with attributes for sender,receiver, and amount. Implement a method within the class to transfer money from the sender's account to the receiver's account..		
1D	Implement a function to add new blocks to the miner and dump the blockchain.		
2A	Write a python program to demonstrate mining.		
2B	Demonstrate the use of the Bitcoin Core API to interact with a Bitcoin Core node		
2C	Demonstrating the process of running a blockchain node on your local machine.		
2D	Demonstrate mining using geth on your private network		
3A	Write a Solidity program that demonstrates various types of functions including regular functions, view functions, pure functions, and the fallback function.		
3B	Write a Solidity program that demonstrates function overloading, mathematical functions, and cryptographic functions		
3C	Write a Solidity program that demonstrates various features including contracts, inheritance, constructors, abstract contracts, interfaces.		
3D	Write a Solidity program that demonstrates use of libraries, assembly, events, and error handling.		
4A	Install and demonstrate use of hyperledger-Irhoa		
4B	Demonstration on interacting with NFT		

Practical No.1A

Aim:-Develop a secure messaging system where users can exchange encrypted messages using RSA public-key cryptography.

Code:-

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import serialization, hashes
from cryptography.hazmat.backends import default_backend
import base64
def generate_keys(username):
    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048,
        backend=default_backend()
    )
    public_key = private_key.public_key()
    with open(f"{username}_private.pem", "wb") as f:
        f.write(
            private_key.private_bytes(
                encoding=serialization.Encoding.PEM,
                format=serialization.PrivateFormat.PKCS8,
                encryption_algorithm=serialization.NoEncryption()
            )
        )
    with open(f"{username}_public.pem", "wb") as f:
        f.write(
            public_key.public_bytes(
                encoding=serialization.Encoding.PEM,
                format=serialization.PublicFormat.SubjectPublicKeyInfo
            )
        )
    print(f" Keys generated for {username}")
def load_public_key(filename):
    with open(filename, "rb") as f:
        public_key = serialization.load_pem_public_key(
            f.read(),
            backend=default_backend()
)
    return public_key
def load_private_key(filename):
    with open(filename, "rb") as f:
        private_key = serialization.load_pem_private_key(
            f.read(),
            password=None,
            backend=default_backend()
        )
    return private_key
def encrypt_message(message, public_key):
    encrypted = public_key.encrypt(
        message.encode(),
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
    return base64.b64encode(encrypted).decode()
def decrypt_message(encrypted_message, private_key):
    decrypted = private_key.decrypt(
        base64.b64decode(encrypted_message),
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
    return decrypted.decode()
def main():
    print("==== Secure Messaging Using RSA ====\n")
    generate_keys("Alice")
    generate_keys("Bob")
    print("\nAlice sends message to Bob...\n")
    bob_public = load_public_key("Bob_public.pem")
    message = input("Enter message from Alice to Bob: ")
    encrypted_msg = encrypt_message(message, bob_public)
    print("\nEncrypted Message:\n", encrypted_msg)
```

```
    print("\nBob decrypts the message...\n")
    bob_private = load_private_key("Bob_private.pem")
    decrypted_msg = decrypt_message(encrypted_msg, bob_private)
    print("Decrypted Message:\n", decrypted_msg)
if __name__ == "__main__":
    main()
```

Output:-

```
>>>
    ========== RESTART: C:/Users/lenovo/AppData/Local/Programs/Python/Python311/MscIT Pract 1a.py ==========
    ==== Secure Messaging Using RSA ====

     Keys generated for Alice
     Keys generated for Bob

    Alice sends message to Bob...

    Enter message from Alice to Bob: HELLO

    Encrypted Message:
     cgu82yjXefVA8cVwdU4CpQ2qmUzQ/5q0r9XqOKZsDmYOLqCq1Ed7bHIqVLvrL6a8Jeu0xDhjIwp2rAbh19fIbc/1qH07grfVTuL9PCUP75CkgdOtEzq9zrfNEOrr4sspokmV7XT3teVEirHasqAYDnxfK9+mvULJC/
    5GNWL+dimD/UPZbBdYNfhDVaYfg3LopOD9qxVih4HTfneAR44EJnQqm+vDyOfMYHgPtuGfJzxms8wGGySbF1uMuD7MWFUlv7L4Givi6lBSERcCluyIgWEipKkeH4/YG/CpRz8L3P1ghUw6n+qPDXev1lQEQApcCaEOD
    yznRo/GP15+aJhrUw==

    Bob decrypts the message...

    Decrypted Message:
     HELLO
...
```

Practical No.1B

Aim:-Allow users to create multiple transactions and display them in an organised format.

Code:-

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import serialization, hashes
from cryptography.hazmat.backends import default_backend
import datetime
import hashlib
import json
def generate_keys():
    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048,
        backend=default_backend()
    )
    public_key = private_key.public_key()
    return private_key, public_key
def encrypt_message(public_key, message):
    encrypted = public_key.encrypt(
        message.encode(),
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
    return encrypted
def decrypt_message(private_key, encrypted_message):
    decrypted = private_key.decrypt(
        encrypted_message,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
    return decrypted.decode()
class Transaction:
    def __init__(self, sender, receiver, message):
        self.sender = sender
        self.receiver = receiver
        self.message = message
        self.timestamp = str(datetime.datetime.now())
    def to_dict(self):
        return {
            "Sender": self.sender,
            "Receiver": self.receiver,
            "Message": self.message,
            "Timestamp": self.timestamp
        }
    def hash_transaction(self):
        tx_string = json.dumps(self.to_dict(), sort_keys=True)
        return hashlib.sha256(tx_string.encode()).hexdigest()
def main():
    print("Secure Messaging using RSA + Blockchain Transactions\n")
    print("Generating RSA keys for User A and User B...\n")
    private_A, public_A = generate_keys()
    private_B, public_B = generate_keys()
    blockchain = []
    while True:
        print("\n1. Send Secure Message")
        print("2. View Transactions")
        print("3. Exit")
        choice = input("Enter choice: ")
        if choice == "1":
            sender = input("Enter Sender Name: ")
            receiver = input("Enter Receiver Name: ")
            message = input("Enter Message: ")
            encrypted_msg = encrypt_message(public_B, message)
            decrypted_msg = decrypt_message(private_B, encrypted_msg)
            tx = Transaction(sender, receiver, decrypted_msg)
            blockchain.append(tx)
            print("\nMessage Sent Securely!")
            print("Encrypted Message:", encrypted_msg.hex()[:60], "...")
     elif choice == "2":
```

```
            print("\nBlockchain Transactions:\n")
            for i, tx in enumerate(blockchain):
                print(f"Transaction {i+1}")
                print("Sender   :", tx.sender)
                print("Receiver :", tx.receiver)
                print("Message  :", tx.message)
                print("Timestamp:", tx.timestamp)
                print("Hash     :", tx.hash_transaction())
                print("-" * 50)
        elif choice == "3":
            print("Exiting...")
            break
    else:
            print("Invalid choice!")
if __name__ == "__main__":
    main()
```

Output:-

```
========== RESTART: C:/Users/lenovo/AppData/Local/Programs/Python/Python311/HscIT Pract 1b.py ==========
Secure Messaging using RSA + Blockchain Transactions

Generating RSA keys for User A and User B...


1. Send Secure Message
2. View Transactions
3. Exit
Enter choice: 1
Enter Sender Name: SHRUTI
Enter Receiver Name: SHRADDHA
Enter Message: HELLO

Message Sent Securely!
Encrypted Message: a6680900006c6a758e81236d0b38365386881310 7ce8c135bca97277d9d4e ...

1. Send Secure Message
2. View Transactions
3. Exit
Enter choice: 2

Blockchain Transactions:

Transaction 1
Sender : SHRUTI
Receiver : SHRADDHA
Message : HELLO
Timestamp: 2026-02-16 00:49:19.773442
Hash : c9e2013d83bc8b2fb588d73e23d51bc7cc97062c2bccc688062ba01cbedd9e3
--------------------------------------------------

1. Send Secure Message
2. View Transactions
3. Exit
Enter choice: 3
Exiting...
>>>
```

```
========== RESTART: C:/Users/lenovo/AppData/Local/Programs/Python/Python311/HscIT Pract 1b.py ==========
Secure Messaging using RSA + Blockchain Transactions

Generating RSA keys for User A and User B...


1. Send Secure Message
2. View Transactions
3. Exit
Enter choice: 1
Enter Sender Name: SHRUTI
Enter Receiver Name: SHRADDHA
Enter Message: HELLO

Message Sent Securely!
Encrypted Message: a6683300006c6a758e81236d0b3836538688131 07ce8c135bca97277d9d4e ...

1. Send Secure Message
2. View Transactions
3. Exit
Enter choice: 2

Blockchain Transactions:

Transaction 1
Sender : SHRUTI
Receiver : SHRADDHA
Message : HELLO
Timestamp: 2026-02-16 00:49:19.773442
Hash : c9e2013d83bc8b2fb588d73e23d51bc7cc97062c2bccc688062ba01cbedd9e3
--------------------------------------------------

1. Send Secure Message
2. View Transactions
3. Exit
Enter choice: 3
Exiting...
>>>
```

Practical No.1C

Aim:-Create a Python class named Transaction with attributes for sender,receiver, and amount. Implement a method within the class to transfer money from the sender's account to the receiver's account..

Code:-

```
class Account:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance
    def display_balance(self):
        print(f"{self.name}'s Balance: ₹{self.balance}")
class Transaction:
    def __init__(self, sender, receiver, amount):
        self.sender = sender
        self.receiver = receiver
        self.amount = amount
    def transfer(self):
        # Check if sender has sufficient balance
        if self.sender.balance >= self.amount:
            self.sender.balance -= self.amount
            self.receiver.balance += self.amount
            print(f"\nTransaction Successful!")
            print(f"{self.amount} transferred from {self.sender.name}
                to {self.receiver.name}")
        else:
            print("\n Transaction Failed: Insufficient Balance")
def main():
    acc1 = Account("Alice", 5000)
    acc2 = Account("Bob", 3000)
    print("Initial Balances:")
    acc1.display_balance()
    acc2.display_balance()
    amount = float(input("\nEnter amount to transfer: "))
    transaction = Transaction(acc1, acc2, amount)
    transaction.transfer()
    print("\nUpdated Balances:")
    acc1.display_balance()
    acc2.display_balance()
```

```
if __name__ == "__main__":
    main()
```

Output:-

```
>>>
    = RESTART: C:/Users/lenovo/AppData/Local/Programs/Python/Python311/MscIT Pract 1
    c.py
    Initial Balances:
    Alice's Balance: ₹5000
    Bob's Balance: ₹3000

    Enter amount to transfer: 2000

    Transaction Successful!
    2000.0 transferred from Alice to Bob

    Updated Balances:
    Alice's Balance: ₹3000.0
    Bob's Balance: ₹5000.0
```

Practical No.1D

Aim:-Implement a function to add new blocks to the miner and dump the blockchain.

Code:-

```
import hashlib
import datetime
import json
class Block:
    def __init__(self, index, data, previous_hash):
        self.index = index
        self.timestamp = str(datetime.datetime.now())
        self.data = data
        self.previous_hash = previous_hash
        self.hash = self.calculate_hash()
def calculate_hash(self):
        block_string = json.dumps({
            "index": self.index,
            "timestamp": self.timestamp,
            "data": self.data,
            "previous_hash": self.previous_hash
        }, sort_keys=True)
return hashlib.sha256(block_string.encode()).hexdigest()
class Blockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]
 def create_genesis_block(self):
        return Block(0, "Genesis Block", "0")
def get_latest_block(self):
        return self.chain[-1]
     def add_block(self, data):
        latest_block = self.get_latest_block()
        new_block = Block(
            index=latest_block.index + 1,
            data=data,
            previous_hash=latest_block.hash
        )
        self.chain.append(new_block)
        print(f"\nBlock {new_block.index} Added Successfully!")
def dump_blockchain(self):
        print("\nFull Blockchain Data:\n")
        for block in self.chain:
            print(f"Block Index     : {block.index}")
            print(f"Timestamp       : {block.timestamp}")
            print(f"Data            : {block.data}")
            print(f"Previous Hash   : {block.previous_hash}")
            print(f"Current Hash    : {block.hash}")
            print("-" * 60)
def main():
    blockchain = Blockchain()
    while True:
        print("\n1. Add New Block")
        print("2. Dump Blockchain")
        print("3. Exit")
        choice = input("Enter choice: ")
```

```
        if choice == "1":
            data = input("Enter block data: ")
            blockchain.add_block(data)
        elif choice == "2":
            blockchain.dump_blockchain()
        elif choice == "3":
            print("Exiting...")
            break
        else:
            print("Invalid choice!")
if __name__ == "__main__":
    main()
```

Output:-

```
>>> ========== RESTART: C:/Users/lenovo/Desktop/SY Msc IT/BC Pract 1d.py ==========

1. Add New Block
2. Dump Blockchain
3. Exit
Enter choice: 1
Enter block data: SKC

Block 1 Added Successfully!

1. Add New Block
2. Dump Blockchain
3. Exit
Enter choice: 2

Full Blockchain Data:

Block Index : 0
Timestamp : 2026-02-27 21:21:31.519002
Data : Genesis Block
Previous Hash : 0
Current Hash : None
------------------------------------------------------------
Block Index : 1
Timestamp : 2026-02-27 21:21:41.811016
Data : SKC
Previous Hash : None
Current Hash : None
------------------------------------------------------------

1. Add New Block
2. Dump Blockchain
3. Exit
Enter choice: 3
Exiting...
```

Practical No.2A

Aim:-Write a python program to demonstrate mining.

Code:-

```
import hashlib
import time
class Block:
    def __init__(self, index, data, previous_hash):
        self.index = index
        self.timestamp = time.time()
        self.data = data
        self.previous_hash = previous_hash
        self.nonce = 0
        self.hash = self.calculate_hash()
 def calculate_hash(self):
        block_string = f"{self.index}{self.timestamp}{self.data}
        {self.previous_hash}{self.nonce}"
        return hashlib.sha256(block_string.encode()).hexdigest()
 def mine_block(self, difficulty):
        print(f"\n Mining block {self.index}...")
        start_time = time.time()
        target = "0" * difficulty
        while self.hash[:difficulty] != target:
            self.nonce += 1
            self.hash = self.calculate_hash()
        end_time = time.time()
        print(f" Block {self.index} mined!")
        print(f"Hash     : {self.hash}")
        print(f"Nonce    : {self.nonce}")
        print(f"Time Taken: {end_time - start_time:.2f} seconds")
class Blockchain:
    def __init__(self, difficulty=4):
        self.chain = [self.create_genesis_block()]
        self.difficulty = difficulty
    def create_genesis_block(self):
        return Block(0, "Genesis Block", "0")
    def get_latest_block(self):
        return self.chain[-1]
    def add_block(self, data):
        new_block = Block(
            index=len(self.chain),
            data=data,
            previous_hash=self.get_latest_block().hash
        )
        new_block.mine_block(self.difficulty)
        self.chain.append(new_block)

    def display_chain(self):
        print("\n Blockchain Data:\n")
        for block in self.chain:
            print(f"Block Index   : {block.index}")
            print(f"Timestamp     : {block.timestamp}")
            print(f"Data          : {block.data}")
            print(f"Nonce         : {block.nonce}")
            print(f"Hash          : {block.hash}")
            print(f"Previous Hash : {block.previous_hash}")
            print("-" * 60)
def main():
    print("Blockchain Mining Demonstration\n")
    blockchain = Blockchain(difficulty=4)
    blockchain.add_block("Alice pays Bob 100")
    blockchain.add_block("Bob pays Charlie 50")
    blockchain.add_block("Charlie pays Dave 25")
    blockchain.display_chain()
if __name__ == "__main__":
    main()
```

Output:-

```
 Blockchain Data:

Block Index : 0
Timestamp : 1772257645.3165977
Data : Genesis Block
Nonce : 0
Hash : 1fdf28ab79200e09132e278538ab0f2e36f124a58ee39cb89866d5a646260575
Previous Hash : 0
------------------------------------------------------------
Block Index : 1
Timestamp : 1772257645.3166459
Data : Alice pays Bob 100
Nonce : 12497
Hash : 0000ee6f3d39bee4d24203270f4472112d31c4782c697399c6cabe75c9ace08a
Previous Hash : 1fdf28ab79200e09132e278538ab0f2e36f124a58ee39cb89866d5a646260575
------------------------------------------------------------
Block Index : 2
Timestamp : 1772257645.3413877
Data : Bob pays Charlie 50
Nonce : 16417
Hash : 00001f61eb9c5160e1ffafb685f4299a40589ef438a0b29bcfb41159a71304ec
Previous Hash : 0000ee6f3d39bee4d24203270f4472112d31c4782c697399c6cabe75c9ace08a
------------------------------------------------------------
Block Index : 3
Timestamp : 1772257645.368393
Data : Charlie pays Dave 25
Nonce : 32919
Hash : 00002a78dd5d4ac5fbd7023340d949a455c36431913b67fdbad9998b23ed9ac3
Previous Hash : 00001f61eb9c5160e1ffafb685f4299a40589ef438a0b29bcfb41159a71304ec
------------------------------------------------------------
>>>
```

Practical No.2B

Aim:-Demonstrate the use of the Bitcoin Core API to interact with a Bitcoin Core node

Code:-

Install Bitcoin Core
Download from: https://bitcoincore.org
Run bitcoind (daemon) or bitcoin-qt
use Testnet or Regtest, NOT mainnet.
Enable RPC in bitcoin.conf
Create or edit:
C:\Users\<username>\AppData\Roaming\Bitcoin\bitcoin.conf
Add:
server=1
rpcuser=student
rpcpassword=student123
rpcport=8332
testnet=1
Restart Bitcoin Core after saving.
pip install python-bitcoinrpc'"

```
from bitcoinrpc.authproxy import AuthServiceProxy, JSONRPCException
rpc_user = "student"
rpc_password = "student123"
rpc_port = 18332   # Testnet RPC port

rpc_connection = AuthServiceProxy(
    f"http://{rpc_user}:{rpc_password}@127.0.0.1:{rpc_port}"
)
try:
    print("Bitcoin Core RPC Connected\n")
    blockchain_info = rpc_connection.getblockchaininfo()
    print("Blockchain Info:")
    print("Chain        :", blockchain_info["chain"])
    print("Blocks       :", blockchain_info["blocks"])
    print("Difficulty   :", blockchain_info["difficulty"])
    print("-" * 50)
    network_info = rpc_connection.getnetworkinfo()
    print("Network Info:")
    print("Version      :", network_info["version"])
    print("Connections  :", network_info["connections"])
    print("-" * 50)
    wallet_info = rpc_connection.getwalletinfo()
    print("Wallet Info:")
    print("Balance      :", wallet_info["balance"], "BTC")
    print("Tx Count     :", wallet_info["txcount"])
    print("-" * 50)
    latest_block_hash = rpc_connection.getbestblockhash()
    block = rpc_connection.getblock(latest_block_hash)
    print("Latest Block:")
    print("Block Hash   :", latest_block_hash)
    print("Height       :", block["height"])
    print("Transactions :", len(block["tx"]))
    print("-" * 50)
except JSONRPCException as e:
    print("RPC Error:", e)
except Exception as e:
    print("Connection Error:", e)
```

Practical No.2C

Aim:-Demonstrating the process of running a blockchain node on your local machine.

Code:-

Use testnet or regtest.
Download from the official site:
https://bitcoincore.org
Create bitcoin.conf
Location
C:\Users\<username>\AppData\Roaming\Bitcoin\
Add the following configuration:INI
server=1
testnet=1
daemon=1
txindex=1
rpcuser=student
rpcpassword=student123
rpcport=18332
Option A: Using GUI

Open:
bitcoin-qt
Using Command Line (Recommended for labs)
Open terminal / command prompt and run:
bitcoind -testnet
Expected output:
Bitcoin Core starting
Loading block index...
Done loading
Check node status using bitcoin-cli:
bitcoin-cli -testnet getblockchaininfo
Sample Output

```
{
  "chain": "test",
  "blocks": 342156,
  "headers": 342156,
  "difficulty": 2432898.34,
  "initialblockdownload": false
}
```

Check Network Connections
bitcoin-cli -testnet getnetworkinfo
Sample output:
"connections": 8
Stop the Node Safely
bitcoin-cli -testnet stop
'''

Practical No.2D

Aim:-Demonstrate mining using geth on your private network.

Code:-

Step 1: Install Geth
Download
Official website:
https://geth.ethereum.org
Verify installation
geth version
Expected output:
Geth
Version: 1.xx.x
Step 2: Create Genesis Block (Private Network)
Create a file named genesis.json

```
{
  "config": {
    "chainId": 2026,
    "homesteadBlock": 0,
    "eip150Block": 0,
    "eip155Block": 0,
    "eip158Block": 0
  },
  "difficulty": "1",
  "gasLimit": "8000000",
  "alloc": {}
}
```

Step 3: Initialize Private Blockchain
Create a data directory and initialize it:
geth init genesis.json --datadir privatechain
Expected output:
Successfully wrote genesis state
Step 4: Start Geth Node (Private Network)
```
geth --datadir privatechain \
    --networkid 2026 \
    --http \
    --http.addr 127.0.0.1 \
    --http.port 8545 \
    --http.api eth,net,web3,personal,miner \
    console
```
You will enter the Geth JavaScript console:
>
Step 5: Create Ethereum Account
Inside the Geth console:
personal.newAccount("password")
Sample output:

"0x8f9a3d0b3e0e0b..."
Set this account as coinbase (miner account):

eth.coinbase
If empty:
miner.setEtherbase(eth.accounts[0])
Step 6: Start Mining
miner.start(1)
Output:-
INFO [..] Successfully sealed new block
Blocks are now being mined
Ether is rewarded to miner account

Step 7: Check Mined Blocks & Balance
Check block number
eth.blockNumber
Check balance
eth.getBalance(eth.coinbase)
Convert to Ether:
web3.fromWei(eth.getBalance(eth.coinbase), "ether")
Step 8: Stop Mining
miner.stop()
Sample Mining Output
INFO [06-12|11:35:42] Successfully sealed new block number=1
INFO [06-12|11:35:44] Successfully sealed new block number=2
INFO [06-12|11:35:46] Successfully sealed new block number=3
'''

Practical No.3A

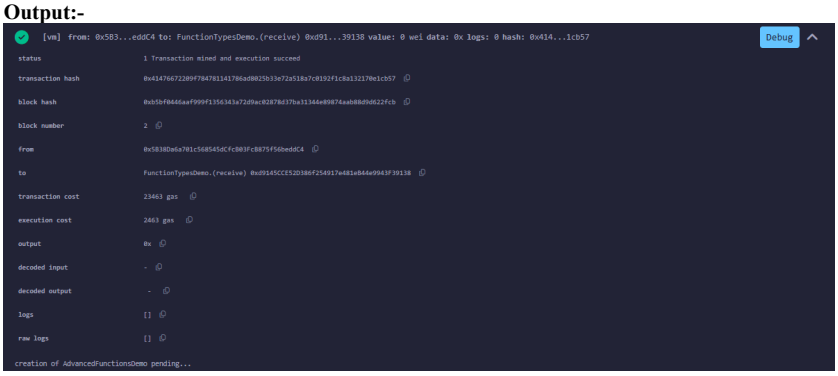
Aim:-Write a Solidity program that demonstrates various types of functions including regular functions, view functions, pure functions, and the fallback function.

Code:-

- Open browser
- Go to: https://remix.ethereum.org
- Create file FunctionTypesDemo.sol
- Paste Solidity code
- Compile & Deploy
- Test functions interactively

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract FunctionTypesDemo {
    uint public number;
    address public lastCaller;
    uint public balance;
    function setNumber(uint _num) public {
        number = _num;
        lastCaller = msg.sender;
    }
function getNumber() public view returns (uint) {
        return number;
    }
function add(uint a, uint b) public pure returns (uint) {
        return a + b;
    }
    receive() external payable {
        balance += msg.value;
    }
fallback() external payable {
        lastCaller = msg.sender;
    }
}
```

Output:-



Practical No.3B

Aim:- Write a Solidity program that demonstrates function overloading, mathematical functions, and cryptographic functions

Code:-

AdvancedFunctionsDemo.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract AdvancedFunctionsDemo {
    function add(uint a, uint b) public pure returns (uint) {
        return a + b;
    }
function add(uint a, uint b, uint c) public pure returns (uint) {
        return a + b + c;
    }
function add(uint a, uint b, uint c, uint d) public pure returns (uint) {
        return a + b + c + d;
    }
function subtract(uint a, uint b) public pure returns (uint) {
        return a - b;
    }
function multiply(uint a, uint b) public pure returns (uint) {
        return a * b;
    }
function divide(uint a, uint b) public pure returns (uint) {
        require(b != 0, "Division by zero not allowed");
```

```
        return a / b;
    }
function modulus(uint a, uint b) public pure returns (uint) {
        return a % b;
    }
function power(uint base, uint exponent) public pure returns (uint) {
        return base ** exponent;
    }
function hashWithKeccak(string memory message) public pure returns (bytes32) {
        return keccak256(abi.encodePacked(message));
    }
function hashWithSHA256(string memory message) public pure returns (bytes32) {
        return sha256(abi.encodePacked(message));
    }
function hashWithRIPEMD160(string memory message) public pure returns (bytes20) {
        return ripemd160(abi.encodePacked(message));
    }
}
```

Steps:

1. Open **Remix IDE**
2. Create AdvancedFunctionsDemo.sol
3. Paste the code
4. Compile with Solidity 0.8.x
5. Deploy the contract
6. Test functions from **Deploy & Run** panel

Output:-

Practical No.3C

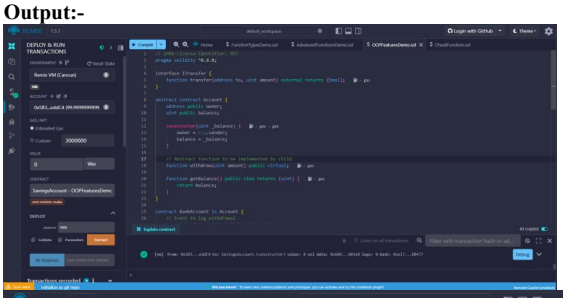
Aim:-Write a Solidity program that demonstrates various features including contracts, inheritance, constructors, abstract contracts, interfaces.

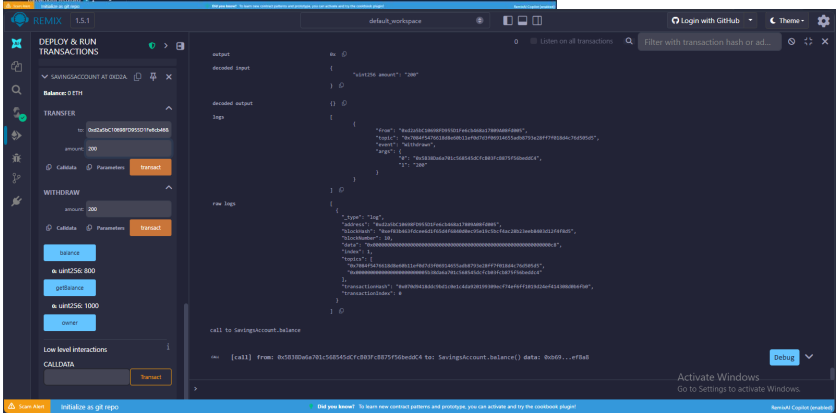
Code:

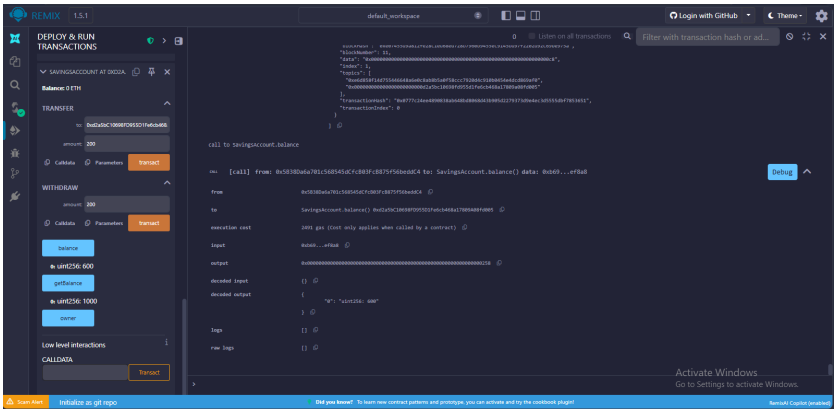
OOPFeaturesDemo.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
interface ITransfer {
    function transfer(address to, uint amount) external returns (bool);
}
abstract contract Account {
    address public owner;
    uint public balance;
    constructor(uint _balance) {
        owner = msg.sender;
        balance = _balance;
    }
    function withdraw(uint amount) public virtual;
    function getBalance() public view returns (uint) {
        return balance;
    }
}
contract BankAccount is Account {
    constructor(uint _balance) Account(_balance) {}
    function withdraw(uint amount) public override {
        require(msg.sender == owner, "Only owner can withdraw");
        require(balance >= amount, "Insufficient balance");
        balance -= amount;
    }
}
contract SavingsAccount is BankAccount, ITransfer {
 constructor(uint _balance) BankAccount(_balance) {}
  function transfer(address to, uint amount) external override returns (bool) {
        require(balance >= amount, "Insufficient balance");
        balance -= amount;
        return true;
    }
}
```

Output:-





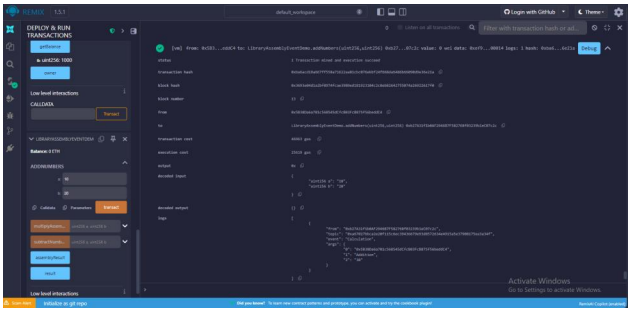


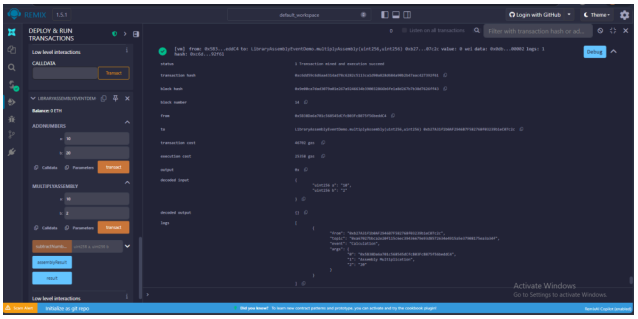
Practical No.3D
Aim:-Write a Solidity program that demonstrates use of libraries, assembly, events, and error handling.

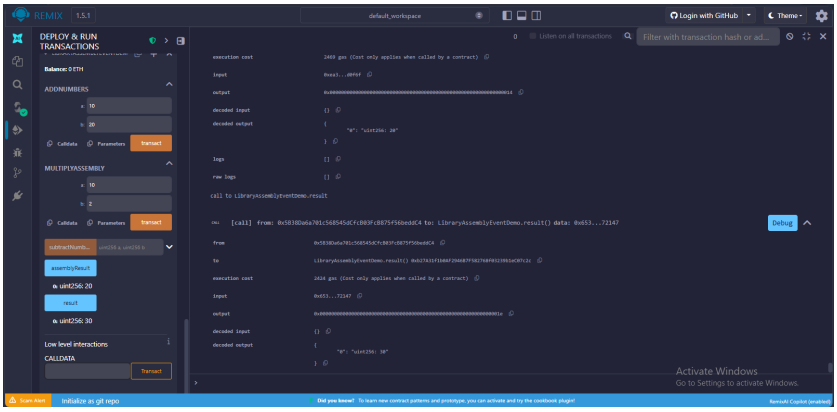
Code:-

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
library MathLibrary {
    function add(uint a, uint b) internal pure returns (uint) {
        return a + b;
    }
function subtract(uint a, uint b) internal pure returns (uint) {
        require(b <= a, "Subtraction underflow");
        return a - b;
    }
}
contract LibraryAssemblyEventDemo {
  using MathLibrary for uint;
  uint public result;
   uint public assemblyResult;
   event Calculation(address indexed user, string operation, uint value);
    error ValueTooSmall(uint value);
    function addNumbers(uint a, uint b) public {
        result = MathLibrary.add(a, b);
        emit Calculation(msg.sender, "Addition", result);
    }

    function subtractNumbers(uint a, uint b) public {
        result = MathLibrary.subtract(a, b);
        emit Calculation(msg.sender, "Subtraction", result);
    }
    function multiplyAssembly(uint a, uint b) public {

        if(a == 0 || b == 0){
            revert ValueTooSmall(0);
        }

        assembly {
            let c := mul(a,b)
            sstore(assemblyResult.slot, c)
        }

        emit Calculation(msg.sender, "Assembly Multiplication", assemblyResult);
    }
}
```

Output:-







Practical No.4A

Aim:-Install and demonstrate use of hyperledger-Irhoa

Code:-

Download Docker Desktop:

https://www.docker.com/products/docker-desktop

Verify installation:

docker --version

Clone Hyperledger Iroha Repository

Open terminal and run:

git clone https://github.com/hyperledger/iroha.git

Move into the directory:

cd iroha

Start Iroha Network

Run the Docker container:

docker-compose up

This command will start:

- Iroha node
- PostgreSQL database
- Supporting services

Check Running Containers

Run:

docker ps

This confirms the Iroha blockchain node is running.

Demonstration – Create User

Use Iroha CLI to create a new user.

Example command:

createAccount user@test

This creates a blockchain account.

Create Asset

Example:

createAsset coin#test

This creates a digital asset on the blockchain.

Transfer Asset

Example transaction:

transferAsset admin@test user@test coin#test 100

Check Account Balance

Run:

getAccountAssets user@test

Practical No.4B

Aim:-Demonstration on interacting with NFT

Code:-

Open **Remix IDE** in a browser.
https://remix.ethereum.org
Create a new Solidity file named:
NFTDemo.sol
Write the NFT smart contract.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract NFTDemo {

   struct NFT {
      uint id;
      string name;
      address owner;
   }

   mapping(uint => NFT) public nfts;
   uint public totalNFTs;

   function mintNFT(string memory _name) public {
      totalNFTs++;
      nfts[totalNFTs] = NFT(totalNFTs, _name, msg.sender);
   }

   function getNFT(uint id) public view returns(string memory, address) {
      NFT memory nft = nfts[id];
      return (nft.name, nft.owner);
   }
}
```

Compile the program using **Solidity Compiler**.
Deploy the contract using **Remix VM**.
Mint a new NFT.
Example input:
mintNFT("DigitalArt")
Check the NFT details.
Example:
getNFT(1)